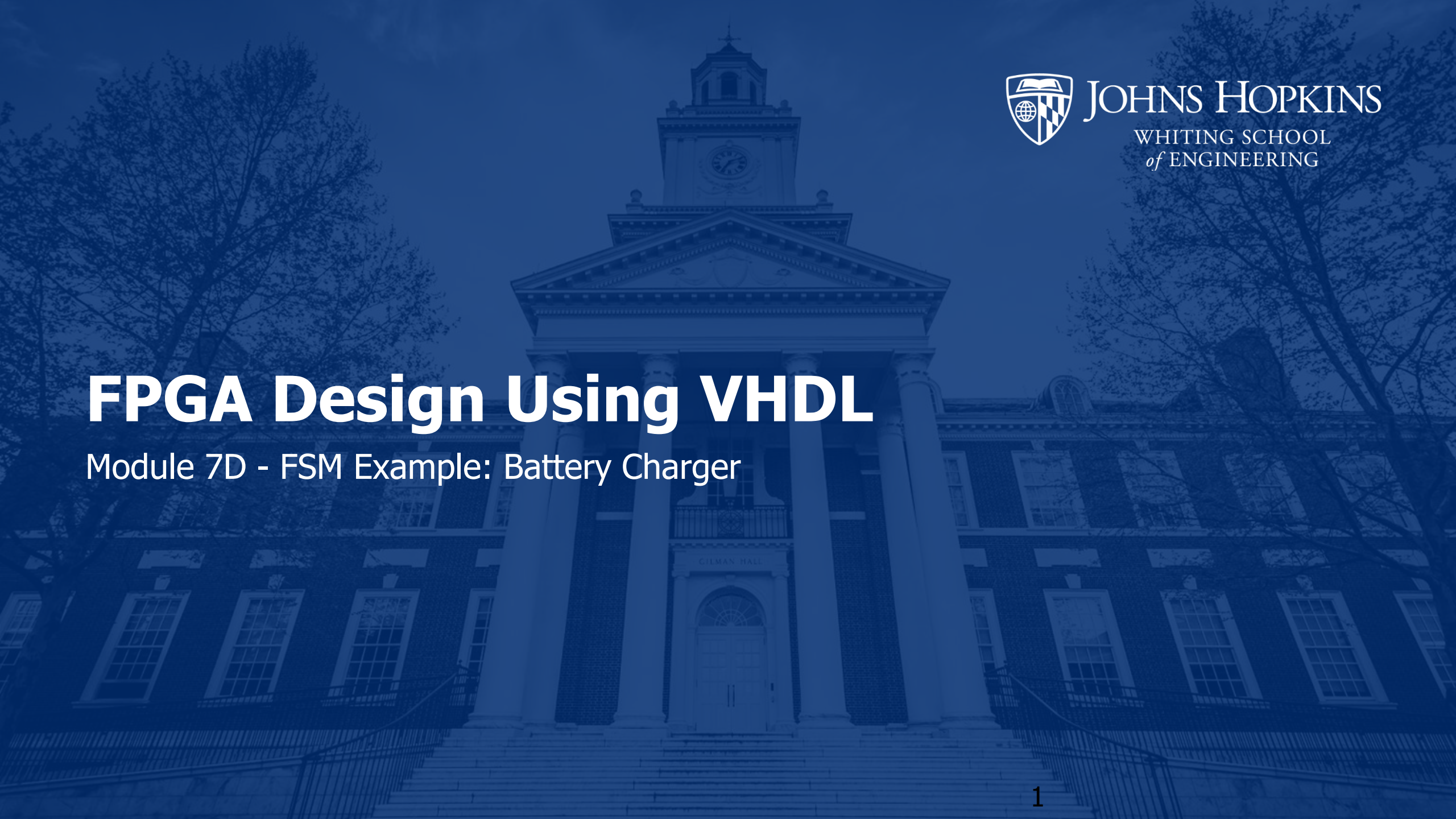


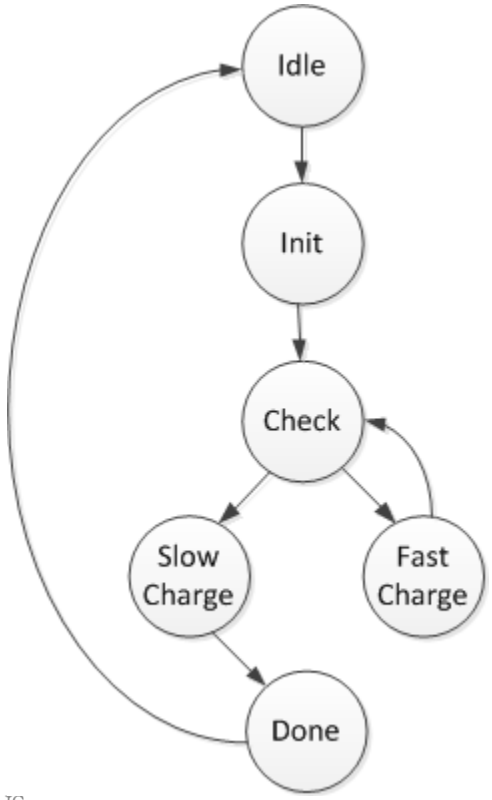
JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

FPGA Design Using VHDL

Module 7D - FSM Example: Battery Charger

Battery Charger: Using FSM

- Consider a "smart" battery charger (which will be simple to design using a FSM)



Start here – no batteries inserted in charger

When batteries inserted, turn on slow charge and remain for two minutes

Check voltage threshold

Charge fast for 10 minutes, then go back and check voltage threshold

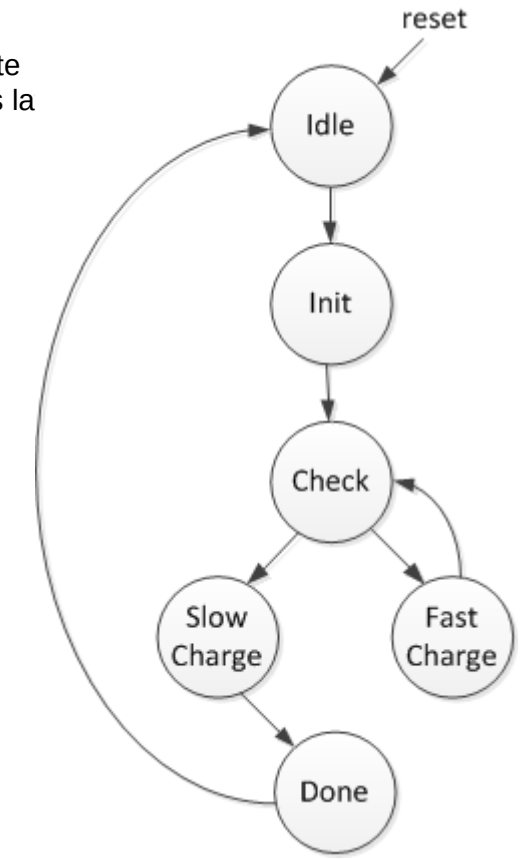
Done charging, wait for battery to be removed

Battery Charger: Example

```
type statetype is (idle, init, check, slowchg, fastchg, donechg);
signal curstate,next_state : statetype;
```

```
state_logic : process( curstate, battery_attached, highc_thresh, doneToFSM)
begin
   next_state <= curstate;
   case curstate is
      when idle =>
         if(battery_attached = '1') then
            next_state <= init;
         end if;
      when init => -- 2 minute initialization charge
         if(doneToFSM = '1') then
            next_state <= check;
         end if;
      when check =>
         if(highc_thresh = '0') then
            next_state <= fastchg;
         else
            next_state <= slowchg;
         end if;
      when fastchg => -- 10 minute fast charge
         if(doneToFSM = '1') then
            next_state <= check;
         end if;
      when slowchg => --  2 minute slow charge
         if(doneToFSM = '1') then
            next_state <= donechg;
         end if;
      when donechg =>
         if(battery_attached = '0') then
            next_state <= idle;
         end if;
   end case;
end process state_logic;
```

Different FSM template
This statement avoids la



Can you tell what FSM this is?

Battery Charger using FSM

```
-- making state register
process(clk50, reset)
begin
   if(reset = '1') then
      curstate <= idle;
   elsif(rising_edge(clk50)) then
      curstate <= next_state;
   end if;
end process;
```

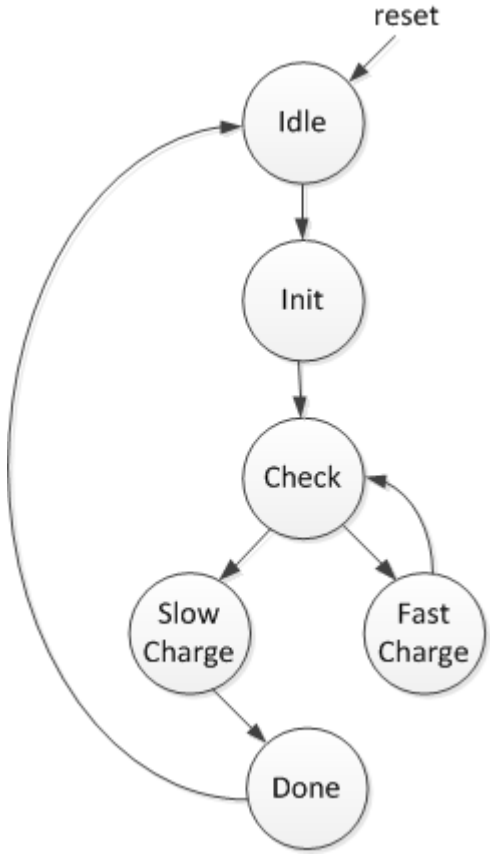
Can you tell what FSM this is?

```
-- timer: 2 minutes or 10 minutes
-- incomplete to save space
timerproc : process(reset, clk50)
begin
    if(reset = '1') then
        timer <= 0;
    elsif(rising_edge(clk50)) then
        -- if timer reaches max count
        -- stop counting and issue a
        -- "done counting" pulse
    end if;
end process timerproc;
```

```
-- making outputs as CSA's
MaxCounter <= x"08F0D180" when ( (curstate = init) or (curstate = slowchg) )
              else x"17D78400" when (curstate = fastchg) else (others => '0');
startFromFSM <= '0' when ((curstate = donechg) or (curstate = idle) or (curstate = check))
                else '1';
incCheckCntr <= '1' when (curstate = check) else '0';
resetCheckCntr <= '1' when (curstate = idle) else '0';
```

Battery Charger: FSM Utilization

- Perform the analyses **manually**
- How many inputs?
  - 3
- How many states?
  - 6
- What type of reset?
  - Asynchronous
- How many state transitions? Tricky!
  - 2 x states = 12
- How many outputs? Tricky!
  - 32



Battery Charger FSM Utilization

- What does Xilinx Vivado synthesis report tell us?

```
Synthesizing Unit <batteryCharger_top>.
Related source file is "/t/BatteryCharger/batteryCharger.vhd".
    Found finite state machine <FSM_1> for signal <curstate>.
    -----------------------------------------------------------------------
    | States             | 6                                              |
    | Transitions        | 12                                             |
    | Inputs             | 3                                              |
    | Outputs            | 31                                             |
    | Clock              | clk50                     (rising_edge)        |
    | Reset              | reset                     (positive)           |
    | Reset type         | asynchronous                                   |
    | Reset State        | idle                                           |
    | Power Up State     | idle                                           |
    | Encoding           | automatic                                      |
    | Implementation     | LUT                                            |
    -----------------------------------------------------------------------
    Found 8-bit up counter for signal <checkCntr>.
    Found 23-bit up counter for signal <dbCntr0>.
    Found 23-bit comparator less for signal <dbCntr0$cmp_lt0000> created at line 50.
    Found 1-bit register for signal <sw_db<0>>.
    Summary:
         inferred   1 Finite State Machine(s).
         inferred   2 Counter(s).
         inferred   1 D-type flip-flop(s).
         inferred   1 Comparator(s).
Unit <batteryCharger_top> synthesized.
```